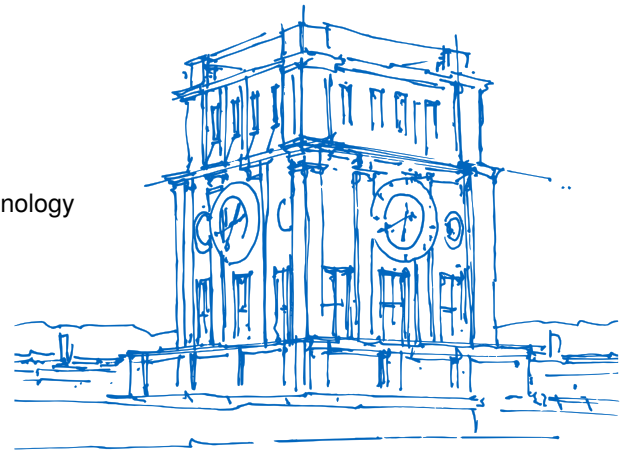
Version Control

Git

Adrian Stein, Thomas Lindae

School of Computation, Information and Technology
Technische Universität München

30. September 2026



Version Control System
3 Primärziele

1. Zugreifen auf frühere Versionen des Quellcodes

2. Gleichzeitiges Arbeiten an verschiedenen features

3. Synchronisation mehrer Entwickler

Git



Linus Torvalds



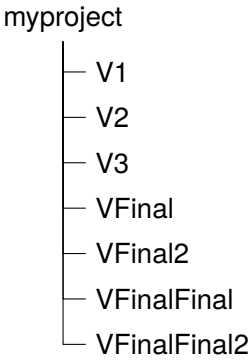
Junio Hamano

Outline

1. Zugreifen auf frühere Versionen des Quellcodes
2. Gleichzeitiges Arbeiten an verschiedenen features
3. Synchronisation mehrer Entwickler

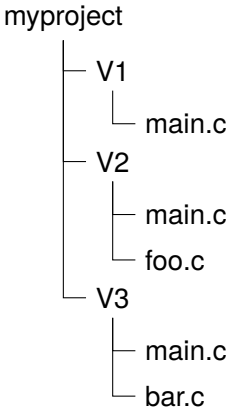
Naiver Ansatz

Wiederholtes speichern des Projekts bei jeder Änderung.



Naiver Ansatz

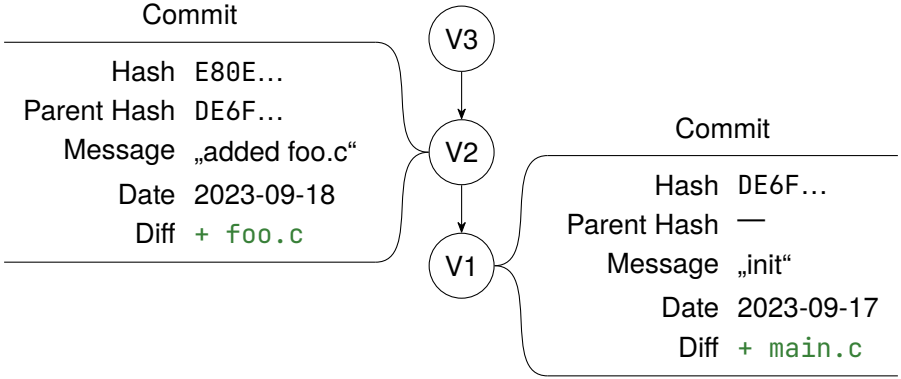
Wiederholtes speichern des Projekts bei jeder Änderung.



Problem: hohe Redundanz und daher großer Speicheraufwand

Git Commits

Idee: Statt immer das ganze Projekt zu speichern, speichert man nur die Änderungen.



Commit Interpretation
Diff vs. Snapshot

Commit als Diff

Ein Commit sind nur die Änderungen zum Parent Commit.

Commit als Snapshot

Ein Commit ist ein Snapshot des Projekts.

Die tatsächliche Implementation von Git speichert Commits nicht als Diffs! Um trotzdem Platzsparend zu sein, werden verschiedene Kompressionsmechanismen verwendet.

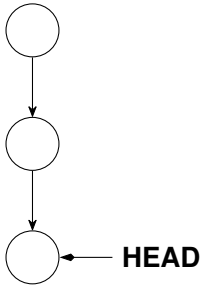
HEAD und Working Tree

Working Tree:

myproject

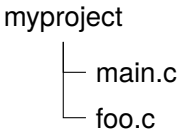
└ main.c

Commit History:



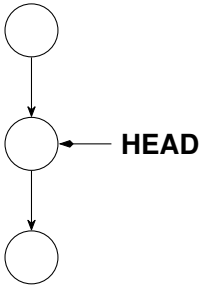
HEAD und Working Tree

Working Tree:



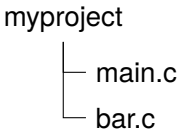

```
$ git switch E80E...
```

Commit History:



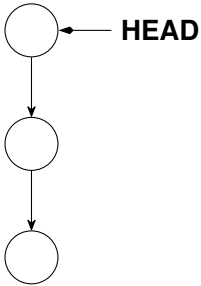
HEAD und Working Tree

Working Tree:

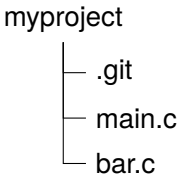



```
$ git switch 2453...
```

Commit History:



Git Repository



`git init`

Der `.git` Ordner speichert alle Informationen die Git braucht, unter anderem:

- Commmits
- HEAD
- Branches
- Remotes

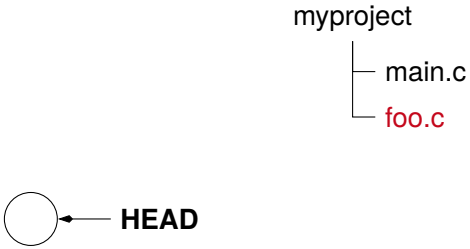
Der Ordner wird von Git verwendet, man sollte ihn also nicht bearbeiten.

Erstellen von Commits

Beim erstellen eines Commits gibt es 3 Stufen:

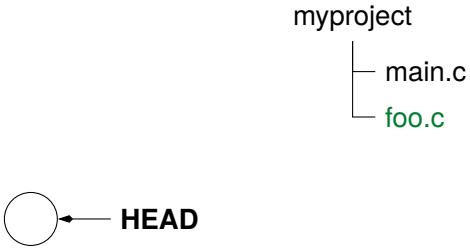
1. Änderungen werden im Working Tree gemacht
2. Änderungen werden „gestaged“: `git add <FILES>`
3. Änderungen werden commited: `git commit -m "<MESSAGE>"`

Erstellen von Commits
Staging



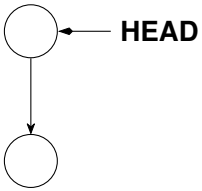

```
$ git status
Untracked files:
  (use "git add <file>...")
        foo.c
```

Erstellen von Commits
Staging




```
$ git status
Untracked files:
  (use "git add <file>...")
        foo.c
$ git add foo.c
$ git status
Changes to be committed:
    new file:   foo.c
```

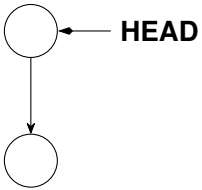
Erstellen von Commits
Staging



myproject
- main.c
- foo.c

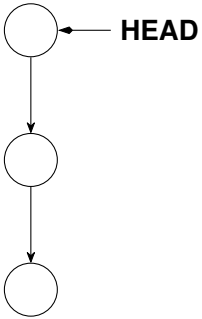
```
$ git status
Untracked files:
  (use "git add <file>...")
        foo.c
$ git add foo.c
$ git status
Changes to be committed:
    new file:   foo.c
$ git commit -m "add foo.c"
```

Erstellen von Commits
Staging



myproject
- main.c
- foo.c
- bar.c

Erstellen von Commits
Staging



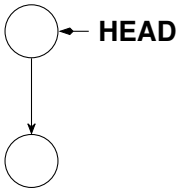
myproject
- main.c
- foo.c
- bar.c

```
$ git commit -m "nothing"
$ git status
Untracked files:
  (use "git add <file>...")
        bar.c
```

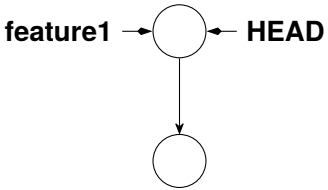
Outline

1. Zugreifen auf frühere Versionen des Quellcodes
2. Gleichzeitiges Arbeiten an verschiedenen features
3. Synchronisation mehrer Entwickler

Git Branch

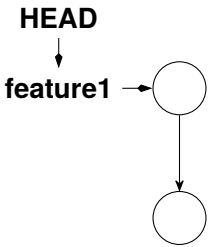


Git Branch



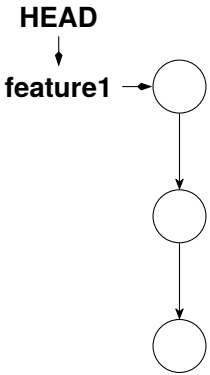

```
$ git branch feature1
```

Git Branch



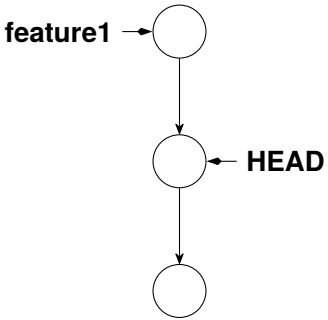

```
$ git branch feature1
$ git switch feature1
```

Git Branch



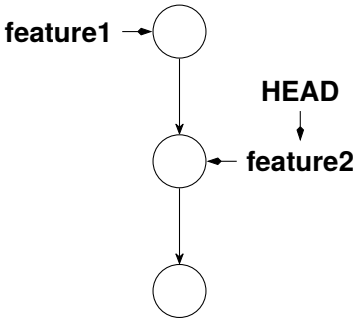

```
$ git branch feature1
$ git switch feature1
$ git commit...
```

Git Branch



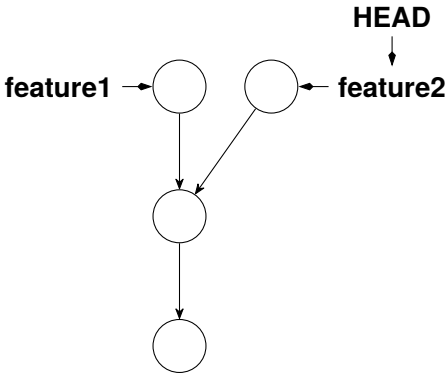

```
$ git branch feature1
$ git switch feature1
$ git commit...
$ git switch...
```

Git Branch




```
$ git branch feature1
$ git switch feature1
$ git commit...
$ git switch...
$ git switch -c feature2
```

Git Branch




```
$ git branch feature1
$ git switch feature1
$ git commit...
$ git switch...
$ git switch -c feature2
$ git commit ...
```

Detached HEAD

Der **HEAD** Pointer ist *detached* wenn er nicht an einem Branch hängt.

Commits die mit einem **detached HEAD** erstellt werden, gehen potenziell verloren!

Branch Interpretation
Pointer vs. Subtree

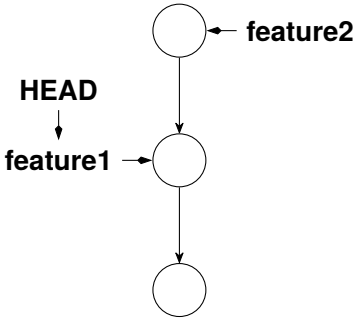
Branch als Pointer
Technische Interpretation

Ein Branch zeigt auf einen einzige Commit.

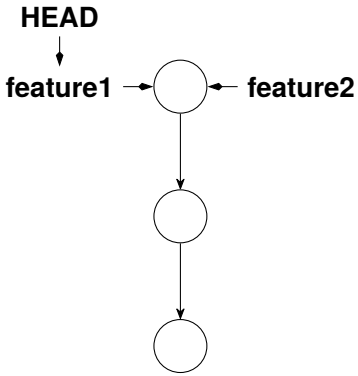
Branch als Subtree
Praktische Interpretation

Ein Branch ist der komplette Teilbaum aller Vorgänger eines Commits.
(Alternativ: Alle Vorgänger bis zur ersten Abzweigung.)

Git Merge
Fast Forward Merge

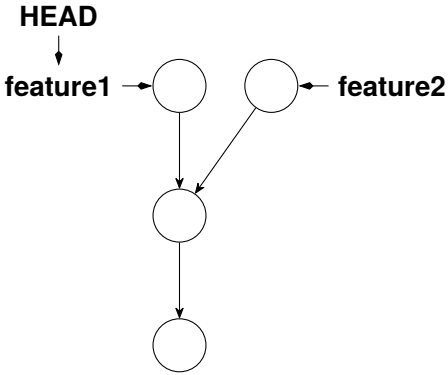


Git Merge
Fast Forward Merge

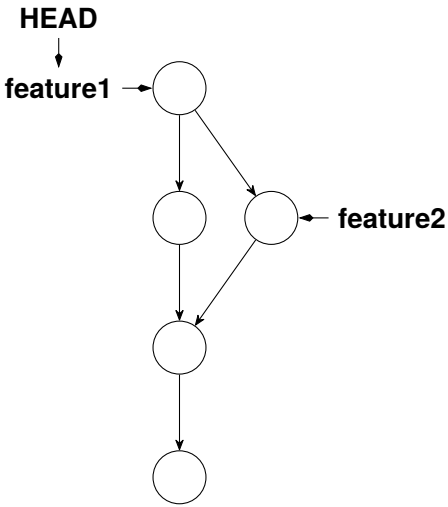



```
$ git merge feature2
```

Git Merge



Git Merge

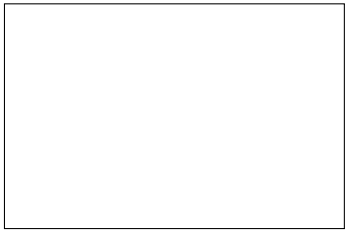



```
$ git merge feature2
```

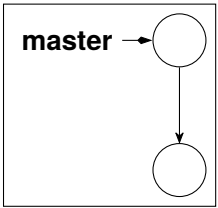
Outline

1. Zugreifen auf frühere Versionen des Quellcodes
2. Gleichzeitiges Arbeiten an verschiedenen features
3. Synchronisation mehrer Entwickler

Git Remotes

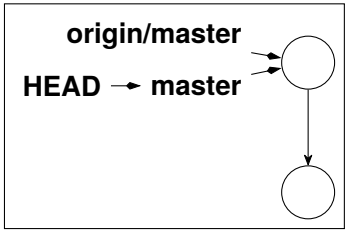


local

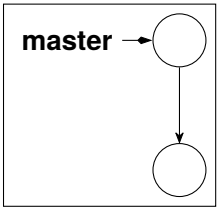


remote

Git Remotes



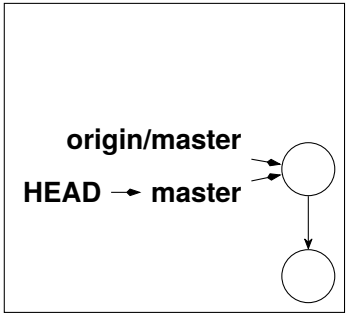
local



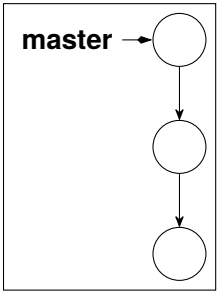
remote

```
$ git clone <REMOTE_URL>
$ git remote
origin
```

Git Fetch

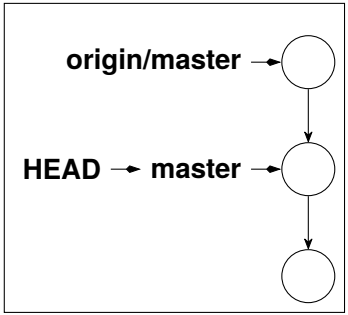


local

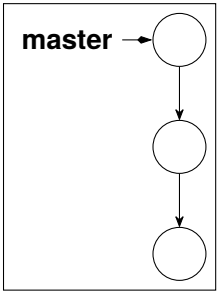


origin

Git Fetch



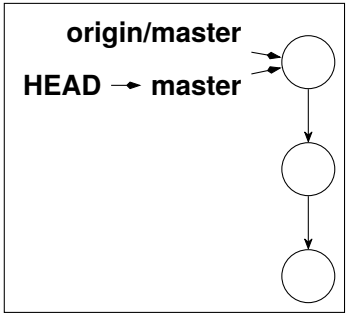
local



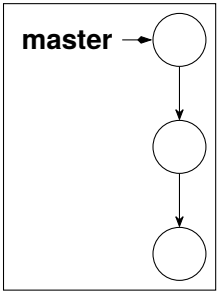
origin

```
$ git fetch
```

Git Fetch



local

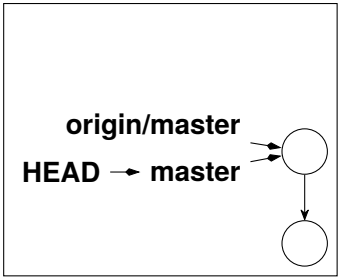


origin

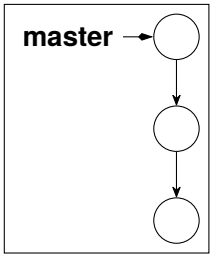
```
$ git fetch
$ git merge origin/master
```

Git Pull

Die Kombination aus `git fetch` und `git merge origin/master` ist so häufig, dass es einen eigenen Command dafür gibt: `git pull`.



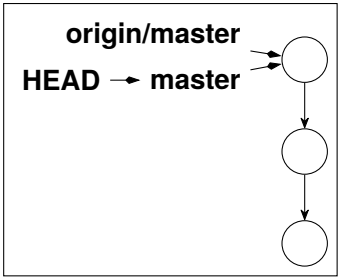
local



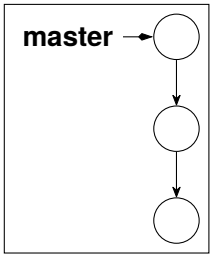
origin

Git Pull

Die Kombination aus `git fetch` und `git merge origin/master` ist so häufig, dass es einen eigenen Command dafür gibt: `git pull`.



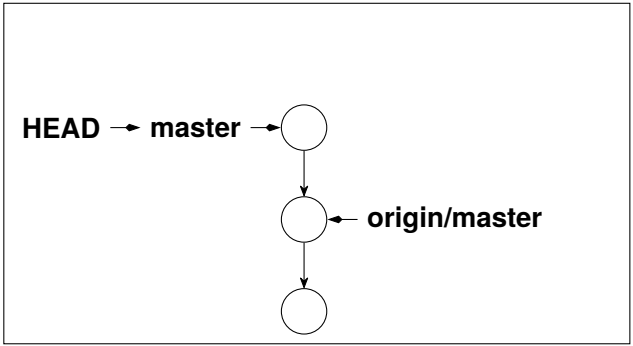
local



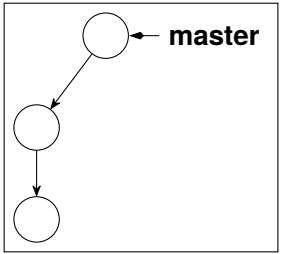
origin

```
$ git pull
```

Git Pull mit Merge Commit

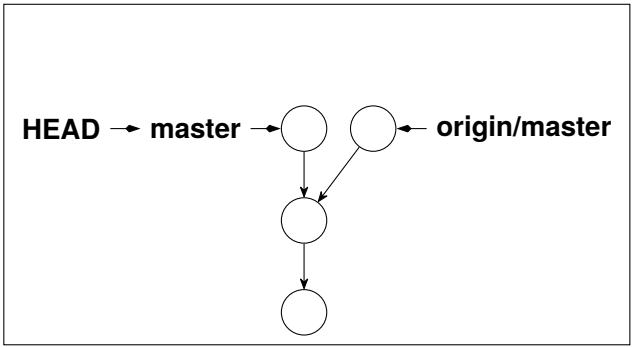


local

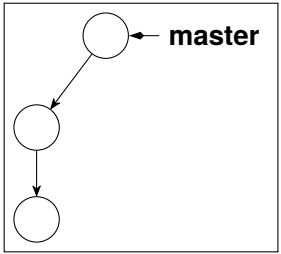


origin

Git Pull mit Merge Commit



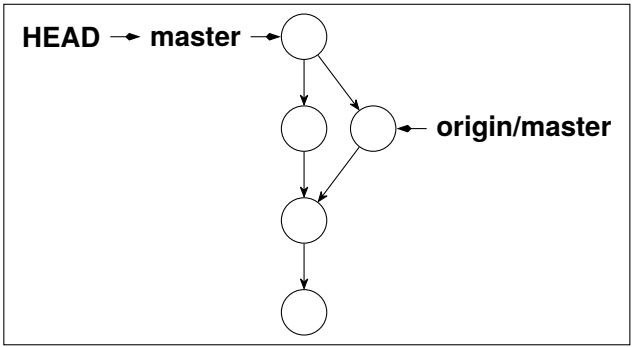
local



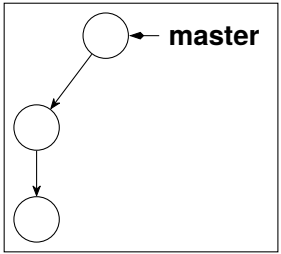
origin

```
$ git fetch
```

Git Pull mit Merge Commit



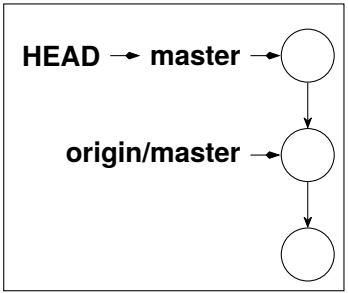
local



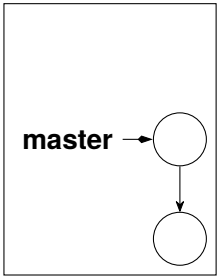
origin

```
$ git fetch
$ git merge origin/master
```

Git Push

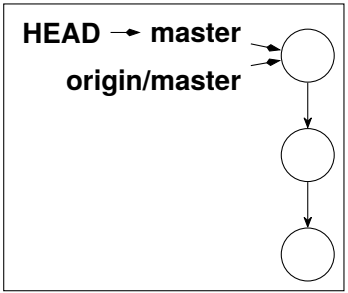


local

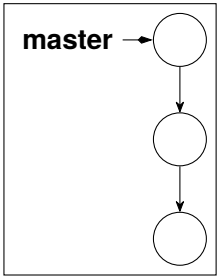


origin

Git Push



local



origin

```
$ git push
```

`git push` erlaubt by default nur Fast-Forward updates!
M. a. W.: `master` muss ein direkter Nachfolger von origin's `master` sein.

Empfehlungen zur Verwendung von Git

Wenn man mit anderen Entwicklern arbeitet, ist die typische Reihenfolge für Änderungen:

`git add` ➔ `git commit` ➔ `git pull` ➔ `git push`

Für komplexe Projekte sollte man sich früh für eine Branch-Ideologie entscheiden,
z. B. Git Flow.

GitHub und GitLab

Für Remote Server kann man entweder einen eigenen Server hosten (mithilfe von z. B. Gitea), oder einen existierenden Service verwenden, wie GitHub oder GitLab.

GitHub:

- 2018 von Microsoft aufgekauft worden
- Der bei weitem verbreiteste Service in der Open Source Welt

GitLab:

- Eigenständig und größtenteils Open Source (oder zumindest source-available)
- Self-Hosting möglich (z. B. auch die LRZ GitLab Instanz)

GitHub und GitLab sind **Services** auf denen man Git Repositories hosten kann, Git selber hat aber nichts mir diesen Services zu tun und kann auch komplett lokal verwendet werden!

Git vergisst nichts

Die Git History ist an sich *unveränderlich*. Es ist zwar möglich die History zu ändern, aber recht umständlich und eigentlich nicht angedacht. Um Änderungen rückgängig zu machen, würde man einen neuen Commit erstellen, der die Änderungen rückgängig macht.

Spätestens wenn Sie einen Commit pushen, können Sie diesen nicht mehr leicht löschen.

Committen Sie keine Secrets wie Passwörter!

Wenn Sie große Dateien committen, sind diese Dateien immer noch in voller Größe vorhanden, auch wenn Sie in einem anderen Commit die Datei wieder löschen!

Git Alternativen

Mercurial

In Verwendung von z. B. Facebook, Mozilla, und das VCS vom TUM-Entwickelten Proof Assistant Isabelle.

Apache Subversion | SVN

Google Piper

Google's internes VCS zum verwalten eines 2+ Milliarden Zeilen Code Monorepository.